

Chapter 22: File Management

Chapter 13: Systems

Programming (Unix Book)

Instructor: Dr. Md Mahfuzur Rahman
Department of Computer Science, GSU

Note: These slides are updated based on original notes
from Raj Sunderraman, Michael Weeks, and Yuan Long

Lab Announcement

- **Do NOT** submit AI-generated code. You will receive **0 credit** for it.
 - Even providing a separate description does not free you from responsible use of AI
 - Use AI only to learn, clarify the concepts, not to produce creditable solutions.
- If the grader thinks the submitted code is not yours, you might be called for an additional interview.
- To receive credit, you must have **the ability to reproduce** the same or similar solution in the interview and/or explain the related inquiries.

Recap: Structure Memory Alignment

```
// size is 8, 4 + 1, then round to multiple of 4  
(int's size),
```

```
struct stu_a {  
    int i;  
    char c;  
};
```

```
// size is 16, 8 + 1, then round to multiple of 8  
(long's size),
```

```
struct stu_b {  
    long l;  
    char c;  
};
```

```
// size is 24, l (lowercase L) need padding by 4  
before it, then round to multiple of 8 (long's  
size),
```

```
struct stu_c {  
    int i;  
    long l;  
    char c;  
};
```

```
// size is 16, 8 + 4 + 1, then round to multiple of  
8 (long's size),
```

```
struct stu_d {  
    long l;  
    int i;  
    char c;  
};
```

```
// size is 16, 8 + 4 + 1, then round to multiple of  
8 (double's size),
```

```
struct stu_e {  
    double d;  
    int i;  
    char c;  
};
```

```
// size is 24, d need align to 8, then round to  
multiple of 8 (double's size),
```

```
struct stu_f {  
    int i;  
    double d;  
    char c;  
};
```

```
// size is 4,  
struct stu_g {  
    int i;  
};
```

```
// size is 8,  
struct stu_h {  
    long l;  
};
```

File Management

- How to manage a file?
 - Open -> Read or Write -> Close
- E.g.
 - open a text file for reading

```
FILE    *fp;  
fp = fopen("test.txt", "r");
```
 - read up to n-1 characters from the input stream referenced by file pointer

```
char buff[255]; //n=255  
fgets(buff, 255, (FILE*)fp);  
printf("%s\n", buff );
```

Writing string to a file

- Write the stream to the output stream referenced by file pointer

```
fputs("hello from System-level programming", fp);
```

- Close a file

```
fclose(fp);
```

Example: writing to a file

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    FILE *fp;
    char *filename = "filename.txt";
    char text[] = "system level programming";

    // Open the file in write mode
    fp = fopen(filename, "w");
    if (fp == NULL) {
        printf("Error opening file for writing.\n");
        return 1;
    }
    // Write text to the file
    fprintf(fp, "%s\n", text);
    printf("Text written to the file successfully.\n");

    // Close the file after writing
    fclose(fp);

    return 0;
}
```

Example: reading from a file

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    FILE *fp;
    char *filename = "filename.txt";
    char buffer[100];

    // Open the file in read mode
    fp = fopen(filename, "r");
    if (fp == NULL) {
        printf("File can't be opened.\n");
        return 1;
    }
    // Read the content of the file and display it
    printf("Reading from the file:\n");
    while (fgets(buffer, sizeof(buffer), fp) != NULL) {
        printf("%s", buffer);
    }
    // Close the file after reading
    fclose(fp);
    return 0;
}
```

File may not be opened:

- If it doesn't exist
- Disk is full
- Invalid path
- File permission error, etc.

File Opening Modes

Text files modes

"r"	Open for reading
"w"	Open for writing (file need not exist/overwrites (truncates) if exists)
"a"	Open for appending (file need not exist/appends if exists)
"r+"	Open for reading and writing, starting at beginning (overwrites w/o truncation)
"w+"	Open for reading and writing (truncate if file exists)
"a+"	Open for reading and writing (append if file exists/goes at the end)

Binary files modes

"rb"	Open for reading
"wb"	Open for writing (file need not exist/overwrites if exists)
"ab"	Open for appending (file need not exist/appends if exists)
"r+b" or "rb+"	Open for reading and writing, starting at beginning
"w+b" or "wb+"	Open for reading and writing (truncate if file exists)
"a+b" or "ab+"	Open for reading and writing (append if file exists)

Systems Programming

- UNIX System calls: C functions that provide access to the file system, processes, and error handling.
- System Calls grouped into 3 main categories:
 - File Management
 - Process Management
 - Error Handling

Error Handling

- function `perror()` describes the system-call error
- `void perror(char* str)`
 - /* standard C function in `stdio.h` */
 - Displays `str` followed by a description of the last system call error.
 - Error 0 is displayed if no error (may show success)

File Management

- System Calls: open, read, write, lseek, unlink, close
- Typical sequence:
 - open
 - read/write
 - close

Standard Library vs. System Calls

- Standard library functions (e.g., `fopen`, `fclose`, `fscanf`, `fprintf`):
 - higher-level, handle buffered I/O, and work with `FILE *` pointers.
 - Optimized performance but no direct control
- System calls (`open`, `read`, `write`, `close`, `read`):
 - lower-level, handle unbuffered I/O, and work with file descriptors (i.e., not with file pointers)
 - provide finer control over file permissions, modes, and flags.
 - less optimized but direct real-time processing

Typical File I/O Sequence

- `int fd;`
 - `/* file descriptor 0: std. in, 1: std. out, 2: std error*/`
- `fd = open(fileName,...);`
- `if (fd == -1) /* deal with error */`
- `read(fd,...) /* read from file */`
- `write(fd,...) /* write to file */`
- `lseek(fd,...) /* seek within file */`
- `close(fd); /* close file */`

Opening a File: `open()`

```
int open(char *fileName, int mode  
[, int permissions])
```

- Allows you to open an existing file or create a new file for r/w
 - **fileName**: absolute or relative path name
 - **mode**: bitwise or-ing of a r/w flag together with zero or more miscellaneous flags (next slide)
 - **Permissions(optional)**: supplied only when file is created (ex. 0600 octal)

Mode Flags

- r/w flags: O_RDONLY, O_WRONLY, O_RDWR
- misc. flags:
 - O_APPEND : position file pointer at the end of file before each write
 - O_CREAT : if the file does not exist, create it
 - O_EXCL : if O_CREAT is set and file exists then open() fails (O_CREAT | O_EXCL are used together, open will create the file **only if it does not already exist.**)
 - O_NONBLOCK : for named pipes (file-like interface for IPC)
 - O_TRUNC : if file exists it is truncated to length 0

Open() Example

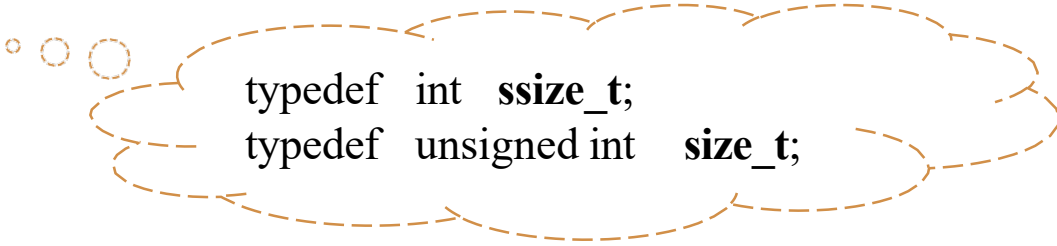
- `tmpfd = open(tmpName, O_CREAT | O_RDWR, 0600);`
 - the file should be created if it doesn't exist
 - it should be opened in read-write mode
 - only read-write permission for the user
- `fd = open (fileName, O_RDONLY);`
 - if the file does not exist or there is no permission to read it), it returns -1.



Read from a regular file: read()

```
ssize_t read(int fd, void *buf,  
             size_t count)
```

- Copies up to `count` bytes from the file referenced by `fd` into buffer pointed to by `buf`
- The bytes are read from current position (maintained by offset/file position) which is then updated accordingly



```
typedef int  ssize_t;  
typedef unsigned int  size_t;
```



Read from a regular file : `read()`

- It returns number of bytes copied
 - returns 0 if it attempts to copy after end of file
 - returns -1 if unsuccessful
- Example:
 - `charsRead = read(fd, buffer, BUFFER_SIZE);`
 - `if (charsRead == 0) break;`
 - `if (charsRead == -1) fatalError();`



Example: read() system call

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>
#include <errno.h>
#include <stdlib.h>
```

read(), compared to
fscanf() works faster
when we read large
blocks of data

```
int main() {
    int fd = open("filename.txt", O_RDONLY); // Opens a file
    if (fd == -1) {
        printf("Error Number %d\n", errno);
        // print error information
        perror("Wrong happend");
        // terminate the Program
        exit(1);
    }
    char buffer[100];
    // Reads from the file using direct system call
    ssize_t bytesRead = read(fd, buffer, sizeof(buffer));
    // Writes to standard output
    write(STDOUT_FILENO, buffer, bytesRead); // STDOUT_FILENO is a constant
    close(fd); // Closes the file
    return 0;
}
```

Output:

Error Number 2

Wrong happened: No such file or directory



Write to a regular file : `write()`

```
ssize_t write(int fd, void *buf,  
              size_t count)
```

- writes the `count` bytes in `buf` to the file referenced by `fd`, and returns the number of bytes written
- The bytes are written into current position (file position or offset) which is then updated accordingly
- If `O_APPEND` was set, file position is set to end of file before each write

Example: writing using write()

```
// add required libraries
int main()
{
    int fd, bytes_written;
    char str_to_write[] = "Computer Science!\n";

    // open the file in write_only mode, create if it doesn't exist, otherwise truncate
    fd = open("unbuffered.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd == -1) {
        // use error code
        exit(1);
    }
    for (int i = 0; i < 1000000; i++) {
        // write N bytes to the file
        bytes_written = write(fd, str_to_write, strlen(str_to_write));
        // Remember: what you would use in case of standard library e.g. fprintf()
        printf("%d bytes were written.\n", bytes_written); }

    close(fd);
    return 0;
}
```

write(), compared to
fprintf(), works
much slower when we
do repetitive small
write operations.



Moving the file pointer: lseek()

```
off_t lseek (int fd, off_t offset,  
             int mode)
```

typedef long **off_t**

- Changes file pointer
- Mode determines how offset is to be used
 - mode = SEEK_SET
 - ✦ offset relative to start of file
 - mode = SEEK_CUR
 - ✦ offset relative to current position of file
 - mode = SEEK_END
 - ✦ offset relative to end of file



Moving the file pointer: `lseek()`

- `lseek` fails if moved before start of file
 - returns new file position if successful
 - returns -1 if unsuccessful
- Example:
 - `lseek(fd, lineStart[i], SEEK_SET);` (`lineStart[i]` is an offset in bytes)
 - `charsRead=read(fd, buffer, lineStart[i+1] - lineStart[i] );`
- To find out current position use:
 - `currentOffset = lseek(fd, 0, SEEK_CUR);`